Problem 1 (Convolutions as Dense Operations):

The goal of this exercise is to better understand the term "parameter-sharing" in the context of convolutional neural networks.

a) Consider the 1-d convolution of an input $\mathbf{x} \in \mathbb{R}^D$ with a kernel $\mathbf{h} \in \mathbb{R}^K$:

$$(\mathbf{x} * \mathbf{h})_k = \sum_{i=1}^{K} x_{K+k-i} h_i \quad \text{for } k \in \{1, \dots, D-K+1\}$$

For fixed $\mathbf{h}$ we can thus view $\mathbf{x} * \mathbf{h}$ as a 1-d convolutional layer in a neural network

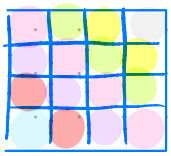
$$f_{\mathbf{h}} : \mathbb{R}^D \to \mathbb{R}^{D-K+1}, \mathbf{x} \mapsto \mathbf{x} * \mathbf{h}$$

Rewrite this operation as a dense layer, namely find a matrix $\mathbf{W_h} \in \mathbb{R}^{(D-K+1)\times D}$ such that you can write

$$f(\mathbf{x}) = \mathbf{W_h x} \quad \forall \mathbf{x}$$

Hint: Use Toeplitz matrix.

Toeplitz matrix = matrix in which every descending diagonal from left to right has identical elements



a) $x \in \mathbb{R}^D$ input

$h \in \mathbb{R}^K$ kernel $\quad D > K$

$$(x * h)_k = \sum_{i=1}^{K} x_{K+k-i} h_i \qquad k \in \{1, 2, \dots, D-K+1\}$$

W_h

$$(x * h)_1 = \sum_{i=1}^{K} x_{K+1-i} h_i =$$

$$= x_{K+1-1} h_1 + x_{K+1-2} h_2 + \dots + x_{K+1-K} h_K =$$

$$= x_K h_1 + x_{K-1} h_2 + \dots + x_1 h_K =$$

$$= h_K x_1 + h_{K-1} x_2 + \dots + h_2 x_{K-1} + h_1 x_K$$

$$W_h x = (x * h)$$

$$\begin{bmatrix} h_K & h_{K-1} & \dots & h_1 & 0 & \dots & 0 \\ 0 & h_K & h_{K-1} & \dots & h_1 & 0 & \\ 0 & 0 & h_K & h_{K-1} & \dots & & \\ & & & & & & \\ 0 & 0 & 0 & h_K & \dots & h_2 & h_1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_K \\ \vdots \\ x_D \end{bmatrix} = \begin{bmatrix} (x*h)_1 \\ (x*h)_2 \\ \vdots \\ (x*h)_{D-K+1} \end{bmatrix}$$

$h_K x_1 + h_{K-1} x_2 + \dots + h_2 x_{K-1} + h_1 x_K$

$$(x * h)_2 = \sum_{i=1}^{K} x_{K+2-i}\, h_i =$$

$$= x_{K+2-1} h_1 + x_{K+2-2} h_2 + \dots + x_{K+2-K} h_K =$$

$$= x_{K+1} h_1 + x_K h_2 + \dots + x_2 h_K =$$

$$= h_K x_2 + h_{K-1} x_3 + \dots + h_2 x_K + h_1 x_{K+1}$$

$$(x * h)_{D-K+1} = \sum_{i=1}^{K} x_{K+D-K+1-i}\, h_i$$
$$x_{D+1-i}$$

$$= x_{D+1-1} h_1 + x_{D+1-2} h_2 + \dots + x_{D+1-K} h_K =$$

$$= x_D h_1 + x_{D-1} h_2 + \dots x_{D-(K-1)} h_K =$$

$$= h_K x_{D-(K-1)} + h_{K-1} x_{D-(K-2)} + \dots + h_2 x_{D-1} + h_1 x_D$$

We can write the 1-d convolution as a matrix–vector product by building a Toeplitz matrix whose rows contain shifted copies of the reversed kernel. Define $\bar{\mathbf{h}} \in \mathbb{R}^K$ by

$$\bar{h}_i := h_{K+1-i}, \qquad i = 1, \dots, K.$$

Then we set $\mathbf{W_h} \in \mathbb{R}^{(D-K+1)\times D}$ as

$$\mathbf{W_h} = \underbrace{\begin{pmatrix} \bar{h}_1 & \bar{h}_2 & \dots & \bar{h}_K & 0 & \dots & 0 \\ 0 & \bar{h}_1 & \dots & \bar{h}_K & \dots & & 0 \\ \vdots & & \ddots & & & \ddots & \vdots \\ 0 & \dots & 0 & \bar{h}_1 & \dots & \bar{h}_K & \end{pmatrix}}_{(D-K+1)\times D}.$$

Equivalently, the k-th row of $\mathbf{W_h}$ is the vector $\bar{\mathbf{h}}$ placed in columns $k, \dots, k+K-1$, and zeros elsewhere. For any $\mathbf{x} \in \mathbb{R}^D$ we then have

$$(\mathbf{W_h x})_k = \sum_{i=1}^{K} \bar{h}_i\, x_{k+i-1} = \sum_{i=1}^{K} h_{K+1-i}\, x_{k+i-1} = \sum_{j=1}^{K} h_j\, x_{K+k-j} = (\mathbf{x} * \mathbf{h})_k,$$

where we used the change of index $j = K + 1 - i$. Hence $f_{\mathbf{h}}(\mathbf{x}) = \mathbf{W_h x}$ for all $\mathbf{x}$, as required.

b) Compare the layer $f_{\mathbf{h}} : \mathbb{R}^D \to \mathbb{R}^{D-K+1}$, $\mathbf{x} \mapsto \mathbf{W_h x}$ with a general dense layer $f : \mathbb{R}^D \to \mathbb{R}^{D-K+1}$, $\mathbf{x} \mapsto \mathbf{W x}$ for some matrix $\mathbf{W} \in \mathbb{R}^{(D-K+1)\times D}$.

What are the number of free parameters in f and in $f_{\mathbf{h}}$? Why does $f_{\mathbf{h}}$ have less free parameters? Think about "parameter-sharing".

$f_h : \mathbb{R}^D \to \mathbb{R}^{D-K+1}$

$x \mapsto W_h x$

$$W_h = \begin{bmatrix} h_K & h_{K-1} & \dots & h_1 & 0 & 0 & 0 \\ 0 & h_K & h_{K-1} & \dots & h_1 & 0 & 0 \\ & & & & & & \\ 0 & 0 & 0 & h_K & h_{K-1} & \dots & h_1 \end{bmatrix}$$

$D-K+1 \times D$

$f : \mathbb{R}^D \to \mathbb{R}^{D-K+1}$

$x \mapsto Wx$

$$W = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1D} \\ w_{21} & w_{22} & \dots & w_{2D} \\ \vdots & & & \vdots \\ w_{D-K+1\,1} & \dots & & w_{D\,D-K+1} \end{bmatrix}$$

$D-K+1 \times D$

$\#\text{params}(f_h) = K$

$\#\text{params}(f) = D(D-K+1)$

A general dense layer is parametrized by an arbitrary matrix $\mathbf{W} \in \mathbb{R}^{(D-K+1)\times D}$. Every entry of $\mathbf{W}$ is an independent learnable parameter, so the total number of free parameters is

$$\#\text{params}(f) = D\,(D-K+1).$$

For the convolutional layer, the matrix $\mathbf{W_h}$ is fully determined by the kernel $\mathbf{h} \in \mathbb{R}^K$: every non-zero entry of $\mathbf{W_h}$ is equal to one of the K entries of $\mathbf{h}$, and these same K numbers are reused (shifted) in each row. Thus

$$\#\text{params}(f_{\mathbf{h}}) = K.$$

We say that convolution uses *parameter sharing*: instead of learning a different set of D weights for each of the $(D-K+1)$ outputs, the same K weights are shared across all spatial positions. This drastically reduces the number of free parameters while still allowing the model to detect the same pattern at different locations.

Problem 2 (Batch Normalization):
Assume we have a fully connected neural network f with input $\mathbf{x} \in \mathbb{R}^d$ described by the recursive system

- $\mathbf{h}^{(0)}(\mathbf{x}) = \mathbf{x} \in \mathbb{R}^d$
- $\mathbf{h}^{(l+1)}(\mathbf{x}) = \mathbf{W}^{(l+1)}\tilde{\mathbf{h}}^{(l)}(\mathbf{x}) \in \mathbb{R}^{d_{l+1}}$ where $\mathbf{W}^{(l+1)} \in \mathbb{R}^{d_{l+1} \times d_l}$
- $\tilde{\mathbf{h}}^{(l+1)}(\mathbf{x}) = \phi\left(\mathbf{h}^{(l+1)}(\mathbf{x})\right) \in \mathbb{R}^{d_{l+1}}$ where $\phi : \mathbb{R} \to \mathbb{R}$ is a non-linearity applied component-wise
- $f(\mathbf{x}) = \left(\mathbf{W}^{(L+1)}\right)^T \tilde{\mathbf{h}}^{(L)}(\mathbf{x}) \in \mathbb{R}$ where $\mathbf{W}^{(L)} \in \mathbb{R}^{d_L}$

We can describe the batch norm operation at layer $l+1$ as follows. Fix a dataset $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$ and take a mini batch $\mathcal{B} = \{\mathbf{x}_1, \ldots, \mathbf{x}_B\}$ which we assume for simplicity is just the first B examples.
During the forward-pass for one step of SGD we calculate the pre-activations $\mathbf{h}^{(l+1)}(\mathbf{x}_1), \ldots, \mathbf{h}^{(l+1)}(\mathbf{x}_B)$. Given those values, we calculate mean and variance statistics as

- $\boldsymbol{\mu}^{(l+1)}(\mathcal{B}) = \frac{1}{B}\sum_{i=1}^{B} \mathbf{h}^{(l+1)}(\mathbf{x}_i) \in \mathbb{R}^{d_{l+1}}$
- $\boldsymbol{\sigma}^{(l+1)}(\mathcal{B}) \in \mathbb{R}^{d_{l+1}}$ such that $\sigma_j^{(l+1)} = \sqrt{\frac{1}{B-1}\sum_{i=1}^{B}\left(h_j^{(l+1)}(\mathbf{x}_i) - \mu_j^{(l+1)}\right)^2}$

Instead of propagating $h^{(l+1)}(\mathbf{x}_i)$ through the non-linearity ϕ for $i = 1, \ldots, B$, we pass

$$\mathbf{z}_j^{(l+1)}(\mathbf{x}) = \gamma_j^{(l+1)} \frac{h_j^{(l+1)}(\mathbf{x}) - \mu_j^{(l+1)}}{\sigma_j^{(l)}} + \beta_j^{(l+1)} \quad \text{for } j = 1, \ldots, d_{l+1}$$

where $\boldsymbol{\gamma}^{(l+1)}, \boldsymbol{\theta}^{(l+1)} \in \mathbb{R}^{d_{l+1}}$ are learnable parameters. The equations for a layer thus become:

- $\mathbf{h}^{(l+1)}(\mathbf{x}) = \mathbf{W}^{(l+1)}\tilde{\mathbf{h}}^{(l)}(\mathbf{x}) \in \mathbb{R}^{d_{l+1}}$
- $\mathbf{z}^{(l+1)}(\mathbf{x}) = \boldsymbol{\gamma}^{(l+1)} \odot \frac{\mathbf{h}^{(l+1)}(\mathbf{x}) - \boldsymbol{\mu}^{(l+1)}(\mathcal{B})}{\boldsymbol{\sigma}^{(l+1)}(\mathcal{B})} + \boldsymbol{\theta}^{(l+1)} \in \mathbb{R}^{d_{l+1}}$
- $\tilde{\mathbf{h}}^{(l+1)}(\mathbf{x}) = \phi\left(\mathbf{z}^{(l+1)}(\mathbf{x})\right) \in \mathbb{R}^{d_{l+1}}$

It is very important to realize that $\boldsymbol{\mu}^{(l)}(\mathcal{B})$ and $\boldsymbol{\sigma}^{(l)}(\mathcal{B})$ are a function of the batch $\mathcal{B}$ and thus vary in different forward passes, while $\boldsymbol{\gamma}^{(l)}$ and $\boldsymbol{\theta}^{(l)}$ do not.

f : fully connected neural network

$x \in \mathbb{R}^D$: input

$h^{(0)}(x) = x \in \mathbb{R}^d$

$h^{(l+1)}(x) = W^{(l+1)}\,\tilde{h}^{(l)}(x) \in \mathbb{R}^{d_{l+1}}$ where $W^{(l+1)} \in \mathbb{R}^{d_{l+1} \times d_l}$

$\tilde{h}^{(l+1)}(x) = \phi\left(h^{(l+1)}(x)\right) \in \mathbb{R}^{d_{l+1}}$

$\phi : \mathbb{R} \to \mathbb{R}$ is a non-linear function applied to each component

$f(x) = \left(W^{(L+1)}\right)^T \tilde{h}^{(L)}(x) \in \mathbb{R}$ $\quad W^{(L+1)} \in \mathbb{R}^{d_L}$

$\{x_1, \ldots, x_n\} \subset \mathbb{R}^d$

$B = \{x_1, x_2, \ldots, x_B\}$ mini batch of B examples

With one forward pass for one step of SGD we compute the pre-activations:

$$h^{(l+1)}(x_1),\ h^{(l+1)}(x_2),\ \dots,\ h^{(l+1)}(x_B)$$

We compute the mean

$$\mu^{(l+1)}(B) = \frac{1}{B}\sum_{i=1}^{B} h^{(l+1)}(x_i) \quad \in \mathbb{R}^{d_{l+1}}$$

We compute the standard deviation

$$\sigma^{(l+1)}(B) \in \mathbb{R}^{d_{l+1}}$$

$$\sigma_j^{(l+1)} = \sqrt{\frac{1}{B-1}\sum_{i=1}^{B}\left(h_j^{(l+1)}(x_i) - \mu_j^{(l+1)}\right)^2}$$

Instead of passing $h^{(l+1)}(x_i)$ through ϕ for $i = 1 \dots B$ we pass $z^{(l+1)}(x_i)$

$$z_j^{(l+1)}(x_i) = \gamma_j^{(l+1)} \frac{h_j^{(l+1)}(x_i) - \mu_j^{(l+1)}}{\sigma_j^{(l+1)}} + \beta_j^{(l+1)}$$

$$j = 1, \dots, d_{l+1}$$

$\gamma^{(l+1)}, \beta^{(l+1)} \in \mathbb{R}^{d_{l+1}}$ are learnable parameters.

For a layer

$$h^{(l+1)}(x) = w^{(l+1)}\,\tilde{h}^{(l)}(x) \in \mathbb{R}^{d_{l+1}}$$

$$z^{(l+1)}(x) = \gamma^{(l+1)} \odot \frac{h^{(l+1)}(x) - \mu^{(l+1)}(B)}{\sigma^{(l+1)}(B)} + \beta^{(l+1)} \in \mathbb{R}^{d_{l+1}}$$

$$\tilde{h}^{(l+1)}(x) = \phi\left(z^{(l+1)}(x)\right) \in \mathbb{R}^{d_{l+1}}$$

b) Why do we even rescale again with $\gamma^{(l)}$? To that end, consider a very simple neural network

$$f(x) = w^{(2)}\phi(w^{(1)}x)$$

where $x \in \mathbb{R}$, $w^{(1)} \in \mathbb{R}$ and $w^{(2)} \in \mathbb{R}$. What happens to the representation power of f if we apply batch norm on the first layer without applying the rescale operation?

$$f(x) = w^{(2)}\,\phi\left(w^{(1)}x\right)$$

$x \in \mathbb{R}$, $w^{(1)} \in \mathbb{R}$ and $w^{(2)} \in \mathbb{R}$

$$h^{(1)} = w^{(1)}x$$

Consider a batch B: $B = \{x_1, x_2 \dots x_B\}$

$$\mu := \frac{1}{B}\sum_{i=1}^{B} x_i$$

$$\sigma := \sqrt{\frac{1}{B-1}\sum_{i=1}^{B}(x_i - \mu)^2}$$

$$h^{(1)}(x_1),\ h^{(1)}(x_2),\ \dots,\ h^{(1)}(x_B)$$

$$\mu^{(1)} = \frac{1}{B}\sum_{i=1}^{B} h^{(1)}(x_i) = \frac{1}{B}\sum_{i=1}^{B} w^{(1)}x_i = \frac{w^{(1)}}{B}\sum_{i=1}^{B} x_i = w^{(1)}\mu$$

$$\sigma^{(1)2} = \frac{1}{B-1}\sum_{i=1}^{B}\left(h^{(1)}(x_i) - \mu^{(1)}\right)^2 = \frac{1}{B-1}\sum_{i=1}^{B}\left(w^{(1)}x_i - w^{(1)}\mu\right)^2 =$$

$$= w^{(1)2}\cdot\frac{1}{B-1}\sum_{i=1}^{B}(x_i - \mu)^2 = \left(w^{(1)}\sigma\right)^2$$

$$\Rightarrow \quad \sigma^{(1)} = \sqrt{\left(w^{(1)}\sigma\right)^2} = \left|w^{(1)}\right|\sigma$$

What happens if we apply batch norm without γ

$$z^{(1)}(x) = \frac{h^{(1)}(x) - \mu^{(1)}}{\sigma^{(1)}} = \frac{w^{(1)}x - w^{(1)}\mu}{|w^{(1)}|\sigma} = \operatorname{sgn}(w^{(1)}) \cdot \frac{x-\mu}{\sigma}$$

$$|w^{(1)}| = w^{(1)} \cdot \operatorname{sgn}(w^{(1)})$$

$$\begin{cases} 1 & w^{(1)} \geq 0 \\ -1 & w^{(1)} < 0 \end{cases}$$

$$f(x) = w^{(2)}\phi\left(z^{(1)}(x)\right) = w^{(2)}\phi\left(\operatorname{sgn}(w^{(1)})\ \frac{x-\mu}{\sigma}\right)$$

The magnitude (scale) of $w^{(1)}$ completely disappears, only the sign remains.

If we include $\gamma^{(1)}$

$$z^{(1)}(x) = \gamma^{(1)}\ \frac{h^{(1)}(x) - \mu^{(1)}}{\sigma^{(1)}} = \gamma^{(1)}\ \operatorname{sgn}(w^{(1)})\ \frac{x-\mu}{\sigma}$$

$$f(x) = w^{2}(x)\ \phi\left(\gamma^{(1)}\ \operatorname{sgn}(w^{(1)}) \cdot \frac{x-\mu}{\sigma}\right)$$

For this network we have $h^{(1)}(x) = w^{(1)}x$. Consider a batch $\mathcal{B} = \{x_1, \ldots, x_B\}$ of scalar inputs, and denote their empirical mean and standard deviation by

$$\mu := \frac{1}{B}\sum_{i=1}^{B} x_i, \qquad \sigma := \sqrt{\frac{1}{B-1}\sum_{i=1}^{B}(x_i - \mu)^2}.$$

The corresponding pre-activations then have

$$\mu^{(1)} = \frac{1}{B}\sum_{i=1}^{B} h^{(1)}(x_i) = \frac{1}{B}\sum_{i=1}^{B} w^{(1)}x_i = w^{(1)}\mu,$$

and

$$(\sigma^{(1)})^2 = \frac{1}{B-1}\sum_{i=1}^{B}(h^{(1)}(x_i) - \mu^{(1)})^2 = \frac{1}{B-1}\sum_{i=1}^{B}(w^{(1)}x_i - w^{(1)}\mu)^2 = (w^{(1)})^2\sigma^2.$$

Hence $\sigma^{(1)} = |w^{(1)}|\sigma$. If we apply batch normalization *without* a learnable rescaling factor, the normalized pre-activation is

$$\hat{h}^{(1)}(x) = \frac{h^{(1)}(x) - \mu^{(1)}}{\sigma^{(1)}} = \frac{w^{(1)}x - w^{(1)}\mu}{|w^{(1)}|\sigma} = \operatorname{sgn}(w^{(1)})\,\frac{x-\mu}{\sigma}.$$

The network then computes

$$f_{\text{normalized}}(x) = w^{(2)}\phi\Big(\hat{h}^{(1)}(x)\Big) = w^{(2)}\phi\Big(\operatorname{sgn}(w^{(1)})\frac{x-\mu}{\sigma}\Big).$$

We see that the magnitude of $w^{(1)}$ disappears entirely from the model: only its sign remains. Thus many different values of $w^{(1)}$ correspond to exactly the same function $f_{\text{normalized}}$, which effectively reduces the expressive power of the parametrization. If we include a learnable rescaling parameter $\gamma^{(1)}$ in batch norm, the normalized pre-activation becomes

$$\tilde{h}^{(1)}(x) = \gamma^{(1)}\frac{h^{(1)}(x) - \mu^{(1)}}{\sigma^{(1)}} = \gamma^{(1)}\operatorname{sgn}(w^{(1)})\frac{x-\mu}{\sigma}.$$

The network now computes

$$f_{\text{batch-norm}}(x) = w^{(2)}\phi\Big(\tilde{h}^{(1)}(x)\Big) = w^{(2)}\phi\Big(\tilde{\gamma}\,\frac{x-\mu}{\sigma}\Big),$$

where we define $\tilde{\gamma} := \gamma^{(1)}\operatorname{sgn}(w^{(1)})$. Since $\tilde{\gamma}$ is trainable, the model recovers a learnable scaling of the normalized input and thus the same kind of expressive power (in terms of how sharply or weakly ϕ is applied) as the original network with $w^{(1)}$ free. In higher-dimensional networks, the vector $\boldsymbol{\gamma}^{(l)}$ plays the same role: batch normalization removes the uncontrolled dependence of the scale on all previous layers, and the learnable rescaling via $\boldsymbol{\gamma}^{(l)}$ restores the ability of each layer to adapt its effective scale where beneficial.

a) Where lies the difference between

- Standardizing the layer and rescaling it with learnable parameters (batch normalization).
- Not modifying the layer at all.

You can argue very informally.

Without batch normalization, the scale of the pre-activations $\mathbf{h}^{(l)}$ at layer l is determined by the entire product of the previous weight matrices and the distribution of the inputs. As training progresses, these scales can drift significantly: small changes in earlier layers may lead to very large or very small activations in deeper layers. This makes optimization harder (gradients may vanish or explode, and the landscape is poorly conditioned). Batch normalization explicitly standardizes the pre-activations at each layer by subtracting a batch mean and dividing by a batch standard deviation, and then reintroduces a learnable scale and shift via $\boldsymbol{\gamma}^{(l)}$ and $\boldsymbol{\theta}^{(l)}$. In this way, the *distribution* of activations is kept more stable across layers and during training, while the model can still learn whatever overall scale is useful through $\boldsymbol{\gamma}^{(l)}$. Informally, batch norm decouples "how large" the activations are from the complicated interaction of all earlier weights, which typically makes optimization much easier.